

Trabajo Práctico N.º 1

Diseño e Implementación de una Unidad Aritmético-Lógica (ALU) en FPGA

Arquitectura de Computadoras

Alumno: Gianluca Ferraris & Fioramontti Martino

Proyecto Vivado: TP1_ALU

Placa: Digilent Basys3 — FPGA Artix-7 xc7a35tcpg236-1

Herramienta: Xilinx/AMD Vivado Design Suite 2026.1

Septiembre de 2026



Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales
Ingeniería en Computación

1. Introducción y objetivos

Este informe documenta el desarrollo del Trabajo Práctico N.º 1 de la materia Arquitectura de Computadoras: el diseño, la verificación funcional y la implementación física de una Unidad Aritmético-Lógica (ALU) parametrizable sobre una placa de desarrollo Digilent Basys3, basada en el FPGA Xilinx Artix-7 xc7a35tcbg236-1.

El enunciado del trabajo exige tres cosas concretas, que estructuran tanto el desarrollo como este informe: (1) implementar la lógica de la ALU en Verilog de forma parametrizable en ancho de datos, (2) validar ese desarrollo mediante un banco de pruebas (testbench) que combine generación de entradas aleatorias con verificación automática del resultado, y (3) simular el diseño con las herramientas de Vivado incluyendo análisis de tiempo (timing), tanto estático como en simulación post-implementación.

A diferencia de un microcontrolador, donde las operaciones de una ALU ya están fijas en el silicio del fabricante, un FPGA permite describir la lógica digital deseada en un lenguaje de descripción de hardware (HDL) y sintetizarla directamente sobre los bloques lógicos reconfigurables del chip. Esto hace que el mismo ejercicio sirva como introducción concreta a todo el flujo de diseño digital: RTL, síntesis, implementación (place & route), generación de bitstream y programación vía JTAG.

El diseño final quedó estructurado en cuatro módulos Verilog (`alu`, `register`, `button_conditioner` y `top`), un archivo de restricciones físicas (`basys3.xdc`) y un testbench autoverificable (`tb_alu`). Todo el código, los fundamentos teóricos detrás de cada decisión de diseño y los resultados obtenidos se detallan en las secciones siguientes.

2. Marco teórico

2.1. La ALU dentro del datapath de un procesador

En la arquitectura clásica de un procesador (modelo de Von Neumann / carga-almacenamiento), el datapath se organiza alrededor de un banco de registros y una Unidad Aritmético-Lógica: los operandos se leen del banco de registros, circulan por la ALU, que ejecuta la operación indicada por un código de operación (opcode) proveniente de la unidad de control, y el resultado vuelve a escribirse en el banco de registros o se utiliza para calcular una dirección de memoria o de salto. La ALU es, en ese sentido, el bloque combinacional central de cualquier CPU: no tiene memoria propia ni conoce el programa que se está ejecutando, simplemente calcula una función de sus entradas en cada instante.

El diseño de este TP reproduce esa separación de responsabilidades a escala reducida: `alu.v` cumple exactamente el rol de la ALU del datapath (puramente combinacional, sin reloj), mientras que `top.v` cumple el rol que en una CPU real cumplirían el banco de registros y la unidad de control (secuencial, sincronizada por reloj, con lógica de habilitación).

2.2. Codificación de las operaciones (opcode)

El opcode de 6 bits utilizado en este TP no es arbitrario: coincide exactamente con el campo **funct** de 6 bits que usa la ISA MIPS para codificar instrucciones de tipo R (aritmético-lógicas entre registros). La siguiente tabla lo muestra:

Operación	Código (bin)	Código (hex)	Funct MIPS equivalente
ADD (suma)	100000	0x20	ADD
SUB (resta)	100010	0x22	SUB
AND	100100	0x24	AND
OR	100101	0x25	OR
XOR	100110	0x26	XOR
NOR	100111	0x27	NOR
SRL (shift lógico der.)	000010	0x02	SRL
SRA (shift aritm. der.)	000011	0x03	SRA

Nota: en MIPS real estas instrucciones tienen además el campo opcode superior en 000000 (indicando tipo R); acá se usa directamente el campo funct de 6 bits como código de operación completo.

Reutilizar una codificación de una ISA real (en vez de inventar una arbitraria) no es solo prolijos: sienta las bases para que, en un trabajo futuro, esta misma ALU pueda integrarse en un datapath MIPS más completo sin tener que rediseñar la tabla de operaciones.

2.3. Lógica combinacional vs. secuencial en Verilog

Verilog distingue dos estilos de bloque procedural con reglas de inferencia de hardware muy distintas. Un bloque `always @(*)`, como el de `alu.v`, se dispara ante cualquier cambio en sus entradas y no contiene

ningún elemento de memoria: sintetiza puertas lógicas puras (lo que la herramienta llama lógica combinacional). Un bloque `always @(posedge i_clock)`, como los de `register.v` y `button_conditioner.v`, solo actualiza sus salidas en el flanco de subida del reloj y sintetiza flip-flops (lógica secuencial), que sí retienen estado entre ciclos.

Dentro de cada estilo, además, corresponde un tipo de asignación distinto: en bloques combinacionales se usa asignación bloqueante (`=`), donde cada línea se evalúa en orden y el valor queda disponible de inmediato para la siguiente línea del mismo bloque — así se ve en el `case` de `alu.v`. En bloques secuenciales se usa asignación no bloqueante (`<=`), donde todas las actualizaciones ocurren en paralelo al final del ciclo de reloj — así se ve en `register.v`. Mezclar ambos estilos dentro del mismo tipo de bloque es una de las fuentes más comunes de bugs de simulación que no se manifiestan de la misma forma en hardware real.

Un detalle de estilo defensivo presente en `alu.v` es la asignación de valores por defecto a `o_result` y `o_carry` antes del `case`. Sin esa asignación, si el `case` no cubriera explícitamente todas las combinaciones posibles del opcode (y no las cubre: 6 bits admiten 64 combinaciones y solo 8 están definidas), la herramienta de síntesis se vería obligada a inferir un **latch** — un elemento de memoria transparente no deseado — para preservar el valor anterior en los casos no contemplados. Los valores por defecto garantizan que la lógica sea 100% combinacional en todos los caminos.

2.4. Metaestabilidad y sincronización de entradas asíncronas

Los botones y switches de la Basys3 son señales completamente asíncronas respecto del reloj interno del FPGA: no hay ninguna garantía de que un cambio de estado del botón respete los tiempos de setup y hold del primer flip-flop que lo capture. Si la transición ocurre demasiado cerca del flanco de reloj, ese flip-flop puede entrar en un estado **metaestable**: un voltaje intermedio, ni 0 ni 1 franco, que tarda un tiempo variable en resolverse a un valor válido — y ese valor puede propagarse de forma distinta a distintas partes del circuito, generando comportamiento inconsistente.

La mitigación estándar, implementada en `button_conditioner.v` mediante `sync_ff1` y `sync_ff2`, es un **sincronizador de dos flip-flops en cascada**: el primero puede volverse metaestable, pero se le da un ciclo de reloj completo para resolverse antes de que el segundo flip-flop lo capture. Esto no elimina la metaestabilidad (matemáticamente es imposible eliminarla del todo), pero reduce su probabilidad de propagarse a niveles despreciables para una aplicación como esta.

2.5. Rebote de contactos (debounce) y detección de flancos

Un pulsador mecánico no realiza una transición limpia de 0 a 1: los contactos metálicos rebotan físicamente al cerrarse, generando decenas de transiciones espurias en una ventana de algunos milisegundos. A 100 MHz, cada uno de esos rebotes sería indistinguible de una pulsación real. El módulo `button_conditioner` resuelve esto con un **contador antirrebote**: solo acepta un nuevo valor estable si la señal sincronizada se mantuvo constante durante 2 elevado a `NB_COUNT` ciclos consecutivos (con `NB_COUNT=17`, unos 1.31 ms a 100 MHz), reiniciando el conteo cada vez que detecta un cambio.

Sobre esa señal ya estable, un **detector de flanco ascendente** (comparando el valor actual contra una versión retrasada un ciclo, `stable & ~stable_d`) genera un pulso de un único ciclo de reloj por cada pulsación física, independientemente de cuánto tiempo el usuario mantenga el botón apretado. Ese pulso de un ciclo es justamente la señal de habilitación (`enable`) que usan los registros para capturar datos exactamente una vez por pulsación.

2.6. Detección de acarreo/préstamo mediante extensión de bit

Para detectar el acarreo de una suma (o el préstamo de una resta) sin perder información, `alu.v` extiende ambos operandos en un bit adicional antes de operar: `{1'b0, i_data_a} + {1'b0, i_data_b}` produce un resultado de 9 bits para operandos de 8. Si la suma real de dos números de 8 bits sin signo excede 255, ese noveno bit (`sum_ext[8]`) queda en 1 — es exactamente el bit de acarreo. Para la resta, el mismo truco captura el préstamo (borrow): si A es menor que B, la resta con signo extendido da un resultado negativo, que en complemento a dos se refleja como un 1 en ese bit superior.

2.7. Desplazamientos lógico y aritmético

El desplazamiento lógico a la derecha (SRL, operador `>>`) rellena con ceros por la izquierda, sin importar el signo del operando. El desplazamiento aritmético (SRA, operador `>>>`) en cambio replica el bit de signo (el bit más significativo) en las posiciones que quedan libres, preservando el signo del número en complemento a dos. En Verilog, el operador `>>>` solo produce shift aritmético real si su operando izquierdo está explícitamente marcado como signado mediante `$signed(...)`; sin esa conversión, el compilador lo trataría igual que `>>` y el error solo se manifestaría al operar con valores negativos, un bug silencioso y difícil de detectar sin un caso de test dirigido específicamente a ese escenario.

2.8. Flujo de diseño en FPGA: síntesis vs. implementación

Entre el código Verilog y el bitstream final hay dos etapas conceptualmente distintas. La **síntesis** traduce el RTL a una netlist de compuertas lógicas genéricas, mapeadas a los primitivos disponibles en el Artix-7 (LUTs, flip-flops, cadenas de acarreo dedicadas, etc.), pero todavía sin ninguna ubicación física asignada dentro del chip; el análisis de tiempo en esta etapa es una estimación basada en modelos de retardo típicos. La **implementación** toma esa netlist y realiza place (asigna cada elemento lógico a una celda física concreta del silicio) y route (conecta esas celdas usando la matriz de interconexión programable real del FPGA). Recién después de la implementación existen retardos de propagación reales, específicos de esa ubicación física.

2.9. Análisis estático de tiempo (STA)

El análisis estático de tiempo (Static Timing Analysis) recorre todos los caminos posibles entre elementos secuenciales del diseño y verifica, para cada uno, dos condiciones sobre el flip-flop de destino: la condición de **setup** (el dato debe estar estable un tiempo mínimo antes del flanco de reloj) y la de **hold** (debe permanecer estable un tiempo mínimo después del flanco). El margen sobrante en el peor caso se llama slack; si es negativo, el diseño no cumple la frecuencia de reloj especificada. Los indicadores clave

de un reporte de timing en Vivado son el **WNS** (Worst Negative Slack), el **WHS** (Worst Hold Slack) y el **WPWS** (Worst Pulse Width Slack). Un WNS negativo indica una violación de timing real que puede hacer fallar el diseño en hardware aunque simule perfecto en el simulador ideal.

Es importante notar que no todos los caminos de un diseño deben analizarse contra el reloj: señales genuinamente asíncronas (como los switches y botones de este TP) o puramente combinacionales hacia un dispositivo sin requisitos de temporizado (como los LEDs) se excluyen del análisis con `set_false_path`, tal como se hace en `basys3.xdc`. Omitir esta exclusión generaría violaciones de timing artificiales sobre caminos que en la práctica no necesitan cumplir ninguna relación temporal con el reloj, ocultando entre el ruido las violaciones reales que sí importan.

2.10. Verificación funcional: testbench autoverificable

Un testbench autoverificable (self-checking) es aquel que decide por sí mismo, mediante código, si cada resultado obtenido es correcto. El mecanismo central para lograrlo es un **modelo de referencia**: una segunda implementación de la misma especificación funcional, escrita de forma independiente al diseño bajo prueba (DUT). Es un punto sutil pero crítico: si el modelo de referencia fuera una copia literal del `case` del DUT, cualquier error conceptual en la especificación se replicaría idéntico en ambos lados y el testbench pasaría igual sin detectar nada — el modelo debe expresar qué se espera que haga cada operación, no cómo la implementa el diseño.

La cobertura de un testbench se completa combinando dos estrategias complementarias: **casos dirigidos**, elegidos a mano para forzar situaciones límite donde suelen aparecer bugs (overflow de signo, acarreo exacto en el límite, desplazamiento máximo, opcodes fuera de la tabla), y **estímulos aleatorios**, que exploran el espacio de entradas de forma no sesgada. Ninguna de las dos por sí sola es suficiente: los casos dirigidos no tienen por qué cubrir todo el espacio, y el azar puro rara vez golpea justo los valores límite.

Bit lógico	Switch físico	Pin
i_switches[0]	SW0	V17
i_switches[1]	SW1	V16
i_switches[2]	SW2	W16
i_switches[3]	SW3	W17
i_switches[4]	SW4	W15
i_switches[5]	SW5	V15
i_switches[6]	SW6	W14
i_switches[7]	SW7	W13

Botones:

Señal lógica	Botón físico	Pin	Función
i_btn_reset	BTNC (centro)	U18	Reset síncrono (nivel)
i_btn_a	BTNU (arriba)	T18	Cargar operando A
i_btn_b	BTNL (izquierda)	W19	Cargar operando B
i_btn_op	BTNR (derecha)	T17	Cargar código de operación
i_btn_calc	BTND (abajo)	U17	Disparar el cálculo (habilita reg. salida)

LEDs (resultado y banderas):

Señal	LED	Pin
o_leds[0]	LD0	U16
o_leds[1]	LD1	E19
o_leds[2]	LD2	U19
o_leds[3]	LD3	V19
o_leds[4]	LD4	W18
o_leds[5]	LD5	U15
o_leds[6]	LD6	U14
o_leds[7]	LD7	V14
o_led_carry	LD14	P1
o_led_zero	LD15	L1

4. Código fuente comentado

Esta sección reproduce el código fuente completo de cada módulo, tal como quedó en el repositorio del proyecto, seguido de una explicación de las decisiones de diseño más relevantes de cada uno.

4.1. alu.v — Unidad Aritmético-Lógica (combinacional)

Módulo puramente combinacional, sin reloj ni estado propio. No conoce switches, botones ni LEDs: esa responsabilidad es exclusiva de `top.v`. Es genérico en ancho de datos (`NB_DATA`) y en ancho de opcode (`NB_OP_CODE`), lo que permite reutilizarlo con buses más anchos sin modificar una sola línea de su lógica interna.

alu.v — código completo

```
module alu
#(
    parameter NB_DATA      = 8,
    parameter NB_OP_CODE   = 6
)
(
    output reg [NB_DATA-1:0] o_result,
    output reg               o_carry,    // acarreo (ADD) / borrow (SUB)
    output wire              o_zero,     // 1 si el resultado es cero
    input wire [NB_DATA-1:0] i_data_a,
    input wire [NB_DATA-1:0] i_data_b,
    input wire [NB_OP_CODE-1:0] i_op_code
);

    localparam [NB_OP_CODE-1:0] OP_ADD = 6'b100000;
    localparam [NB_OP_CODE-1:0] OP_SUB = 6'b100010;
    localparam [NB_OP_CODE-1:0] OP_AND = 6'b100100;
    localparam [NB_OP_CODE-1:0] OP_OR  = 6'b100101;
    localparam [NB_OP_CODE-1:0] OP_XOR = 6'b100110;
    localparam [NB_OP_CODE-1:0] OP_NOR = 6'b100111;
    localparam [NB_OP_CODE-1:0] OP_SRL = 6'b000010;
    localparam [NB_OP_CODE-1:0] OP_SRA = 6'b000011;

    localparam NB_SHIFT = $clog2(NB_DATA);
    wire [NB_SHIFT-1:0] shamt = i_data_a[NB_SHIFT-1:0];

    wire [NB_DATA:0] sum_ext = {1'b0, i_data_a} + {1'b0, i_data_b};
    wire [NB_DATA:0] sub_ext = {1'b0, i_data_a} - {1'b0, i_data_b};

    always @(*)
    begin
        o_result = {NB_DATA{1'b0}};
        o_carry  = 1'b0;

        case (i_op_code)
            OP_ADD: begin
                o_result = sum_ext[NB_DATA-1:0];
                o_carry  = sum_ext[NB_DATA];
            end
            OP_SUB: begin
                o_result = sub_ext[NB_DATA-1:0];
                o_carry  = sub_ext[NB_DATA];
            end
            OP_AND: o_result = i_data_a & i_data_b;
            OP_OR:  o_result = i_data_a | i_data_b;
            OP_XOR: o_result = i_data_a ^ i_data_b;
            OP_NOR: o_result = ~(i_data_a | i_data_b);
            OP_SRL: o_result = i_data_b >> shamt;
            OP_SRA: o_result = $signed(i_data_b) >>> shamt;
        endcase
    end
endmodule
```

```

        default: begin
            o_result = {NB_DATA{1'b0}};
            o_carry = 1'b0;
        end
    endcase
end

assign o_zero = (o_result == {NB_DATA{1'b0}});
endmodule

```

Puntos clave

- `localparam` en vez de literales sueltos en el `case`: si el enunciado cambiara algún código de operación, se corrige en un único lugar.
- `NB_SHIFT = $clog2(NB_DATA)`: calcula automáticamente cuántos bits hacen falta para expresar el desplazamiento (3 bits para 8, 5 para 32); escribir el rango a mano (`i_data_a[2:0]`) funcionaría solo para 8 bits y rompería en silencio con cualquier otro ancho.
- El bit de desplazamiento (`shamt`) se toma de `i_data_a`, según la convención elegida para este diseño (el operando A funciona como cantidad de desplazamiento en las operaciones de shift).
- Los valores por defecto antes del `case` evitan la inferencia de latches para los 56 códigos de operación no definidos (ver Sección 2.3).

4.2. register.v — Registro genérico con habilitación

Módulo secuencial genérico, reutilizado cuatro veces dentro de `top.v` (para A, B, opcode y salida) en vez de escribir cuatro bloques `always` casi idénticos. Concentra en un único lugar toda la lógica de carga con habilitación y reset síncrono.

register.v — código completo

```

module register
#(
    parameter NB = 8
)
(
    input wire      i_clock,
    input wire      i_reset,    // reset síncrono, activo alto
    input wire      i_enable,
    input wire [NB-1:0] i_data,
    output reg [NB-1:0] o_data
);

always @(posedge i_clock)
begin
    if (i_reset)
        o_data <= {NB{1'b0}};
    else if (i_enable)
        o_data <= i_data;
    // sin else: si i_enable=0 el registro conserva su valor.
end

endmodule

```

Puntos clave

- La ausencia de una rama `else` final no infiere un latch: dentro de un bloque `always @ (posedge i_clock)`, la retención implícita es exactamente el comportamiento nativo de un flip-flop, no una inferencia especial de la herramienta.
- El parámetro `NB` permite reutilizar el mismo módulo con anchos distintos: 8 bits para A/B/opcode y 10 bits (`NB_DATA+2`) para el registro de salida, que empaqueta resultado + carry + zero en un solo vector.
- El reset es síncrono (evaluado solo en el flanco de reloj), no asíncrono: esto evita introducir un segundo camino de reloj/control asíncrono en el diseño, que complicaría el análisis de tiempo.

4.3. button_conditioner.v — Sincronizador, antirrebote y detector de flanco

Resuelve, en un único módulo, los tres problemas que presenta cualquier pulsador mecánico conectado a un circuito síncrono: metaestabilidad, rebote de contactos y duración indefinida del apretón (ver Secciones 2.4 y 2.5 para el marco teórico completo).

button_conditioner.v — código completo

```
module button_conditioner
#(
    parameter NB_COUNT = 17      // 2^17 ciclos @100MHz ~= 1.31 ms
)
(
    input  wire i_clock,
    input  wire i_button,
    output wire o_pulse,
    output wire o_level
);
    // 1) Sincronizador de dos flip-flops (sin reset)
    reg sync_ff1 = 1'b0;
    reg sync_ff2 = 1'b0;

    always @(posedge i_clock)
    begin
        sync_ff1 <= i_button;
        sync_ff2 <= sync_ff1;
    end

    // 2) Antirrebote por contador
    reg [NB_COUNT-1:0] counter = {NB_COUNT{1'b0}};
    reg                stable  = 1'b0;

    always @(posedge i_clock)
    begin
        if (sync_ff2 != stable)
        begin
            counter <= counter + 1'b1;
            if (&counter)
            begin
                stable <= sync_ff2;
                counter <= {NB_COUNT{1'b0}};
            end
        end
        else
            counter <= {NB_COUNT{1'b0}};
    end
end
```

```
// 3) Detector de flanco ascendente
reg stable_d = 1'b0;

always @(posedge i_clock)
    stable_d <= stable;

assign o_pulse = stable & ~stable_d;
assign o_level = stable;
endmodule
```

Puntos clave

- La cadena `sync_ff1` a `sync_ff2` deliberadamente no tiene reset: agregarle uno reintroduciría, sobre la señal de reset misma, el mismo problema de asincronía que este bloque busca resolver.
- El contador antirrebote usa reducción AND (`&counter`) para detectar todos unos de forma compacta, evitando comparar contra una constante explícita del ancho de `NB_COUNT` bits.
- El contador se reinicia (no solo se detiene) apenas la señal sincronizada vuelve a coincidir con `stable`: cualquier rebote que revierta la transición antes de completar los ciclos hace que el conteo empiece de cero.
- Separar `o_pulse` (para habilitar registros, un ciclo) de `o_level` (para reset, mientras se mantiene apretado) evita usar la señal equivocada según el caso.

4.4. top.v — Integración estructural

Módulo de nivel superior: instancia los cinco `button_conditioner`, los cuatro `register`, la `alu`, aplica la máscara de opcode y conecta todo con los puertos físicos declarados en `basys3.xdc`.

top.v — código completo

```
module top
#(
    parameter NB_DATA      = 8,
    parameter NB_OP_CODE   = 6,
    parameter NB_DEBOUNCE = 17
)
(
    input  wire          i_clock,
    input  wire [NB_DATA-1:0] i_switches,
    input  wire          i_btn_reset,
    input  wire          i_btn_a,
    input  wire          i_btn_b,
    input  wire          i_btn_op,
    input  wire          i_btn_calc,
    output wire [NB_DATA-1:0] o_leds,
    output wire          o_led_carry,
    output wire          o_led_zero
);
    wire pulse_a, pulse_b, pulse_op, pulse_calc;
    wire reset;

    button_conditioner #(.NB_COUNT(NB_DEBOUNCE)) u_cond_a (
        .i_clock(i_clock), .i_button(i_btn_a),
        .o_pulse(pulse_a), .o_level()
    );
    button_conditioner #(.NB_COUNT(NB_DEBOUNCE)) u_cond_b (
        .i_clock(i_clock), .i_button(i_btn_b),
```

```

        .o_pulse(pulse_b), .o_level()
    );
    button_conditioner #(.NB_COUNT(NB_DEBOUNCE)) u_cond_op (
        .i_clock(i_clock), .i_button(i_btn_op),
        .o_pulse(pulse_op), .o_level()
    );
    button_conditioner #(.NB_COUNT(NB_DEBOUNCE)) u_cond_calc (
        .i_clock(i_clock), .i_button(i_btn_calc),
        .o_pulse(pulse_calc), .o_level()
    );
    button_conditioner #(.NB_COUNT(NB_DEBOUNCE)) u_cond_reset (
        .i_clock(i_clock), .i_button(i_btn_reset),
        .o_pulse(), .o_level(reset)
    );

    wire [NB_DATA-1:0] reg_a_out, reg_b_out, reg_op_out;

    register #(.NB(NB_DATA)) u_reg_a (
        .i_clock(i_clock), .i_reset(reset), .i_enable(pulse_a),
        .i_data(i_switches), .o_data(reg_a_out)
    );
    register #(.NB(NB_DATA)) u_reg_b (
        .i_clock(i_clock), .i_reset(reset), .i_enable(pulse_b),
        .i_data(i_switches), .o_data(reg_b_out)
    );
    register #(.NB(NB_DATA)) u_reg_op (
        .i_clock(i_clock), .i_reset(reset), .i_enable(pulse_op),
        .i_data(i_switches), .o_data(reg_op_out)
    );

    localparam [NB_DATA-1:0] OP_CODE_MASK =
        { {(NB_DATA-NB_OP_CODE){1'b0}}, {(NB_OP_CODE){1'b1}} };

    wire [NB_DATA-1:0] op_masked = reg_op_out & OP_CODE_MASK;
    wire [NB_OP_CODE-1:0] op_code = op_masked[NB_OP_CODE-1:0];

    wire [NB_DATA-1:0] alu_result;
    wire                alu_carry, alu_zero;

    alu #(.NB_DATA(NB_DATA), .NB_OP_CODE(NB_OP_CODE)) u_alu (
        .o_result(alu_result), .o_carry(alu_carry), .o_zero(alu_zero),
        .i_data_a(reg_a_out), .i_data_b(reg_b_out), .i_op_code(op_code)
    );

    wire [NB_DATA+1:0] alu_bundle = {alu_zero, alu_carry, alu_result};
    wire [NB_DATA+1:0] out_bundle;

    register #(.NB(NB_DATA+2)) u_reg_out (
        .i_clock(i_clock), .i_reset(reset), .i_enable(pulse_calc),
        .i_data(alu_bundle), .o_data(out_bundle)
    );

    assign o_leds      = out_bundle[NB_DATA-1:0];
    assign o_led_carry = out_bundle[NB_DATA];
    assign o_led_zero  = out_bundle[NB_DATA+1];
endmodule

```

Puntos clave

- La máscara de opcode (OP_CODE_MASK) se construye a partir de los parámetros en vez de escribirse como constante literal: con los valores por defecto vale 8'b00111111, pero se recalcula sola si cambian los parámetros.

- El registro de salida cumple doble función: retiene el resultado anterior en los LEDs mientras el usuario modifica los operandos, y además crea deliberadamente un camino flip-flop → lógica combinacional → flip-flop, que es justamente el tipo de camino que el analizador de tiempo de Vivado puede medir.
- Los tres registros de entrada instancian el mismo módulo genérico `register` tres veces en vez de un solo registro de estado ancho, priorizando modularidad por sobre la compacidad del código.

4.5. basys3.xdc — Restricciones físicas y de tiempo

El archivo de restricciones (constraints) cumple dos funciones independientes: asigna cada puerto del módulo `top` a un pin físico concreto del encapsulado del FPGA (`PACKAGE_PIN`), y define el reloj del sistema junto con las exclusiones de análisis de tiempo para las señales genuinamente asíncronas (ver Sección 2.9).

basys3.xdc — extracto (constraints principales)

```
set_property PACKAGE_PIN W5 [get_ports i_clock]
set_property IOSTANDARD LVCMOS33 [get_ports i_clock]
create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [get_ports
i_clock]

## Switches SW0..SW7 -> operandos y codigo de operacion
set_property PACKAGE_PIN V17 [get_ports {i_switches[0]}]
set_property PACKAGE_PIN V16 [get_ports {i_switches[1]}]
set_property PACKAGE_PIN W16 [get_ports {i_switches[2]}]
set_property PACKAGE_PIN W17 [get_ports {i_switches[3]}]
set_property PACKAGE_PIN W15 [get_ports {i_switches[4]}]
set_property PACKAGE_PIN V15 [get_ports {i_switches[5]}]
set_property PACKAGE_PIN W14 [get_ports {i_switches[6]}]
set_property PACKAGE_PIN W13 [get_ports {i_switches[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {i_switches[*]}]

## LEDs LD0..LD7 -> resultado ; LD14 -> carry | LD15 -> zero
set_property PACKAGE_PIN U16 [get_ports {o_leds[0]}]
set_property PACKAGE_PIN E19 [get_ports {o_leds[1]}]
set_property PACKAGE_PIN U19 [get_ports {o_leds[2]}]
set_property PACKAGE_PIN V19 [get_ports {o_leds[3]}]
set_property PACKAGE_PIN W18 [get_ports {o_leds[4]}]
set_property PACKAGE_PIN U15 [get_ports {o_leds[5]}]
set_property PACKAGE_PIN U14 [get_ports {o_leds[6]}]
set_property PACKAGE_PIN V14 [get_ports {o_leds[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_leds[*]}]
set_property PACKAGE_PIN P1 [get_ports o_led_carry]
set_property IOSTANDARD LVCMOS33 [get_ports o_led_carry]
set_property PACKAGE_PIN L1 [get_ports o_led_zero]
set_property IOSTANDARD LVCMOS33 [get_ports o_led_zero]

## btnC (U18) -> reset | btnU (T18) -> cargar A | btnL (W19) -> cargar B
## btnR (T17) -> cargar opcode | btnD (U17) -> calcular
set_property PACKAGE_PIN U18 [get_ports i_btn_reset]
set_property IOSTANDARD LVCMOS33 [get_ports i_btn_reset]
set_property PACKAGE_PIN T18 [get_ports i_btn_a]
set_property IOSTANDARD LVCMOS33 [get_ports i_btn_a]
set_property PACKAGE_PIN W19 [get_ports i_btn_b]
set_property IOSTANDARD LVCMOS33 [get_ports i_btn_b]
set_property PACKAGE_PIN T17 [get_ports i_btn_op]
set_property IOSTANDARD LVCMOS33 [get_ports i_btn_op]
set_property PACKAGE_PIN U17 [get_ports i_btn_calc]
set_property IOSTANDARD LVCMOS33 [get_ports i_btn_calc]
```

```
## Botones y switches son senales ASINCRONAS; LEDs sin requisitos de timing
set_false_path -from [get_ports i_btn_reset]
set_false_path -from [get_ports i_btn_a]
set_false_path -from [get_ports i_btn_b]
set_false_path -from [get_ports i_btn_op]
set_false_path -from [get_ports i_btn_calc]
set_false_path -from [get_ports {i_switches[*]}]
set_false_path -to [get_ports {o_leds[*]}]
set_false_path -to [get_ports o_led_carry]
set_false_path -to [get_ports o_led_zero]

set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]
```

Puntos clave

- `create_clock` es la restricción que habilita todo el análisis de tiempo: sin ella, Vivado no tiene contra qué comparar los retardos de los caminos combinacionales y el reporte de timing quedaría vacío.
- `IOSTANDARD LVCMOS33` en cada puerto declara el nivel de voltaje lógico (3.3 V) con el que opera físicamente ese pin, acorde a la alimentación de la Basys3.
- Cada `set_false_path` excluye explícitamente del análisis de setup/hold a un puerto que no tiene relación temporal real con el reloj del sistema.

5. Testbench (tb_alu.v)

El testbench fue diseñado para instanciar directamente el módulo `alu` (no `top`), evitando toda la complejidad de botones y temporización de la interfaz física, y así poder concentrar la verificación exclusivamente en la corrección funcional de la lógica aritmético-lógica. Cumple los dos requisitos explícitos del enunciado (generación de entradas aleatorias y chequeo automático) y agrega cobertura adicional no exigida: casos borde dirigidos, opcodes inválidos, y verificación de las banderas `carry` y `zero`, no solo del resultado.

5.1. Estructura en tres fases

Fase	Cobertura	Cantidad de casos
1 — Casos borde dirigidos	Overflow con signo, carry exacto, borrow, shifts extremos, operaciones lógicas con patrones conocidos (0x00, 0xFF, 0xAA/0x55)	17
2 — Opcodes inválidos	Ejercita explícitamente la rama default del case en <code>alu.v</code>	5
3 — Estímulos aleatorios	a, b y opcode generados con <code>\$random</code> sobre los 8 opcodes válidos	500

Total: 522 verificaciones. Cada `check()` aplica un estímulo y espera 10 ns antes de comparar.

El mecanismo central del chequeo automático es la tarea `reference_model`, que recalcula el resultado esperado de forma completamente independiente al case del DUT (ver fundamento teórico en la Sección 2.10), y la tarea `check`, que aplica un estímulo, espera la propagación combinacional y compara las tres salidas (`result`, `carry`, `zero`) contra lo calculado por el modelo, incrementando un contador de errores ante cualquier discrepancia.

5.2. Código completo

tb_alu.v — código completo

```
module tb_alu;
    localparam NB_DATA      = 8;
    localparam NB_OP_CODE   = 6;
    localparam NB_SHIFT     = $clog2(NB_DATA);
    localparam N_RANDOM     = 500;

    localparam [NB_OP_CODE-1:0] OP_ADD = 6'b100000;
    localparam [NB_OP_CODE-1:0] OP_SUB = 6'b100010;
    localparam [NB_OP_CODE-1:0] OP_AND = 6'b100100;
    localparam [NB_OP_CODE-1:0] OP_OR  = 6'b100101;
    localparam [NB_OP_CODE-1:0] OP_XOR = 6'b100110;
    localparam [NB_OP_CODE-1:0] OP_NOR = 6'b100111;
    localparam [NB_OP_CODE-1:0] OP_SRL = 6'b000010;
    localparam [NB_OP_CODE-1:0] OP_SRA = 6'b000011;

    reg  [NB_DATA-1:0]  data_a, data_b;
    reg  [NB_OP_CODE-1:0] op_code;
    wire [NB_DATA-1:0]  result;
    wire                carry, zero;

    reg  [NB_DATA-1:0]  exp_result;
    reg                exp_carry, exp_zero;
```

```

reg [NB_OP_CODE-1:0] valid_ops [0:7];
integer i, error_count, check_count;

alu #(.NB_DATA(NB_DATA), .NB_OP_CODE(NB_OP_CODE)) dut (
    .o_result(result), .o_carry(carry), .o_zero(zero),
    .i_data_a(data_a), .i_data_b(data_b), .i_op_code(op_code)
);

// Modelo de referencia: independiente del DUT (ver Sección 2.10)
task automatic reference_model;
    input [NB_DATA-1:0] a, b;
    input [NB_OP_CODE-1:0] op;
    output [NB_DATA-1:0] r;
    output c;
    reg [NB_DATA:0] ext;
    begin
        c = 1'b0;
        case (op)
            OP_ADD: begin ext=(a+b); r=ext[NB_DATA-1:0]; c=ext[NB_DATA]; end
            OP_SUB: begin ext=(a-b); r=ext[NB_DATA-1:0]; c=ext[NB_DATA]; end
            OP_AND: r = a & b;
            OP_OR: r = a | b;
            OP_XOR: r = a ^ b;
            OP_NOR: r = ~(a | b);
            OP_SRL: r = b >> a[NB_SHIFT-1:0];
            OP_SRA: r = $signed(b) >>> a[NB_SHIFT-1:0];
            default: r = {NB_DATA{1'b0}};
        endcase
    end
endtask

task automatic check;
    input [NB_DATA-1:0] a, b;
    input [NB_OP_CODE-1:0] op;
    input [255:0] etiqueta;
    begin
        data_a = a; data_b = b; op_code = op;
        #10;
        reference_model(a, b, op, exp_result, exp_carry);
        exp_zero = (exp_result == {NB_DATA{1'b0}});
        check_count = check_count + 1;
        if (result != exp_result || carry != exp_carry || zero != exp_zero)
            begin
                error_count = error_count + 1;
                $display("[ERROR] t=%0t %0s", $time, etiqueta);
                $display("entradas: op=%b a=%h b=%h", op, a, b);
                $display("obtenido: result=%h carry=%b zero=%b", result,
                    carry, zero);
                $display("esperado: result=%h carry=%b zero=%b", exp_result,
                    exp_carry, exp_zero);
            end
    end
endtask

initial begin
    valid_ops[0]=OP_ADD; valid_ops[1]=OP_SUB; valid_ops[2]=OP_AND;
    valid_ops[3]=OP_OR;
    valid_ops[4]=OP_XOR; valid_ops[5]=OP_NOR; valid_ops[6]=OP_SRL;
    valid_ops[7]=OP_SRA;
    error_count = 0; check_count = 0;

    $display("TP1 - Testbench ALU (NB_DATA=%0d, NB_OP_CODE=%0d)", NB_DATA,
        NB_OP_CODE);

    // -- Fase 1: casos borde dirigidos --
    check(8'hFF, 8'h01, OP_ADD, "ADD con carry out");
    check(8'h7F, 8'h01, OP_ADD, "ADD con overflow con signo");

```

```

check(8'h00, 8'h00, OP_ADD, "ADD cero + cero (flag zero)");
check(8'h00, 8'h01, OP_SUB, "SUB con borrow (A < B)");
check(8'h05, 8'h05, OP_SUB, "SUB resultado cero");
check(8'h01, 8'h80, OP_SRA, "SRA de negativo (0x80 >>> 1)");
check(8'h07, 8'h80, OP_SRA, "SRA maximo (0x80 >>> 7)");
check(8'h01, 8'h7F, OP_SRA, "SRA de positivo (0x7F >>> 1)");
check(8'h01, 8'h80, OP_SRL, "SRL de 0x80 (rellena con cero)");
check(8'h07, 8'hFF, OP_SRL, "SRL maximo (0xFF >> 7)");
check(8'h00, 8'hAA, OP_SRL, "SRL con desplazamiento cero");
check(8'h0F, 8'hF0, OP_NOR, "NOR de complementarios -> 0x00");
check(8'h00, 8'h00, OP_NOR, "NOR de ceros -> 0xFF");
check(8'hFF, 8'hFF, OP_AND, "AND de todos unos");
check(8'hAA, 8'h55, OP_AND, "AND de patrones alternados -> 0x00");
check(8'hAA, 8'h55, OP_OR, "OR de patrones alternados -> 0xFF");
check(8'hFF, 8'hFF, OP_XOR, "XOR de iguales -> 0x00");

// -- Fase 2: opcodes invalidos --
check(8'hAB, 8'hCD, 6'b111111, "opcode invalido 111111");
check(8'hAB, 8'hCD, 6'b000000, "opcode invalido 000000");
check(8'hAB, 8'hCD, 6'b010101, "opcode invalido 010101");
check(8'hAB, 8'hCD, 6'b100001, "opcode invalido 100001 (cercano a ADD)");
check(8'hAB, 8'hCD, 6'b000001, "opcode invalido 000001 (cercano a SRL)");

// -- Fase 3: estímulos aleatorios --
for (i = 0; i < N_RANDOM; i = i + 1)
    check($random, $random, valid_ops[$unsigned($random) % 8], "aleatorio");

if (error_count == 0)
    $display("RESULTADO: OK - pasaron %0d de %0d verificaciones", check_count,
        check_count);
else
    $display("RESULTADO: FALLO - %0d de %0d verificaciones con error",
        error_count, check_count);
$finish;
end
endmodule

```

6. Resultados de simulación

El testbench se ejecutó bajo dos modalidades de simulación distintas dentro de Vivado Simulator (XSim): comportamental (behavioral), contra el RTL puro sin ningún retardo físico, y de timing post-implementación, contra la netlist ya ubicada y ruteada, con los retardos reales anotados mediante un archivo SDF generado automáticamente por Vivado.

6.1. Simulación comportamental (behavioral)

Primera corrida: se cortó a los 1000 ns (límite por defecto de `xsim.simulate.runtime`), antes de completar las 522 verificaciones (que requieren 5220 ns, a razón de 10 ns por cada llamada a `check`). Ajustando ese parámetro a `-all` en Simulation Settings, la corrida completa arrojó:

Salida de la consola Tcl — Run Behavioral Simulation

```
TP1 - Testbench ALU      (NB_DATA=8, NB_OP_CODE=6)

-- Fase 1: casos borde dirigidos --
-- Fase 2: opcodes invalidos (rama default) --
-- Fase 3: 500 estímulos aleatorios --

RESULTADO: OK - pasaron 522 de 522 verificaciones

$finish called at time : 5220 ns : File "tb_alu.v" Line 211
```

El tiempo total (5220 ns = 522 x 10 ns) confirma que las 522 verificaciones se ejecutaron efectivamente, sin cortes intermedios: 0 errores reportados por la tarea `check`.

6.2. Simulación de timing post-implementación

Ejecutada vía Flow Navigator, Simulation, Run Simulation, Run Post-Implementation Timing Simulation, contra `impl_1` ya completo. Vivado generó automáticamente la netlist temporizada y el SDF asociado antes de compilar y correr la simulación:

Salida de la consola Tcl — Run Post-Implementation Timing Simulation

```
TP1 - Testbench ALU      (NB_DATA=8, NB_OP_CODE=6)

-- Fase 1: casos borde dirigidos --
-- Fase 2: opcodes invalidos (rama default) --
-- Fase 3: 500 estímulos aleatorios --

RESULTADO: OK - pasaron 522 de 522 verificaciones

$finish called at time : 5220 ns : File "tb_alu.v" Line 211
```

Nota metodológica

Durante la compilación de esta simulación, Vivado emitió la siguiente advertencia:

Advertencia observada en consola

```
WARNING: [VRFC 10-3609] overwriting previous definition of module 'alu' [alu.v:14]
```

Esto ocurre porque `tb_alu` instancia directamente el módulo `alu` (no `top`), y el fileset de simulación incluye tanto la netlist temporizada generada por Vivado (que sí contiene un `alu` con los retardos reales anotados vía SDF) como el archivo RTL original `alu.v` (sin retardos). Al compartir el mismo nombre de módulo, la segunda definición compilada sobrescribe a la primera, por lo que es probable que el DUT efectivamente simulado en esta corrida haya sido el RTL ideal (retardo cero) y no la versión con retardos físicos anotados. Se documenta esto de forma transparente porque es relevante para interpretar correctamente el alcance de esta verificación: el análisis de tiempo estático (Sección 7), que sí opera exclusivamente sobre el diseño implementado real, sigue siendo la verificación de timing definitiva y válida para este TP. Una simulación de timing post-implementación completamente rigurosa requeriría que el testbench instancie `top` en vez de `alu` directamente, aplicando estímulos por los puertos físicos (switches y botones) — una mejora propuesta para trabajo futuro (ver Sección 9).

6.3. Resumen de resultados

Simulación	Verificaciones	Errores	Tiempo simulado
Behavioral (RTL puro)	522	0	5220 ns
Post-Implementation Timing	522	0	5220 ns

7. Análisis estático de tiempo (Report Timing Summary)

Generado tras completar síntesis e implementación, sobre el diseño `impl_1` ya ubicado y ruteado, con el reloj de 100 MHz definido en `basys3.xdc`.

Métrica	Setup	Hold	Pulse Width
Peor slack (WNS/WHS/WPWS)	4.779 ns	0.172 ns	4.500 ns
Slack negativo total (TNS/THS/TPWS)	0.000 ns	0.000 ns	0.000 ns
Endpoints con violación	0	0	0
Total de endpoints	258	258	137

"All user specified timing constraints are met." — Vivado Report Timing Summary.

Los tres márgenes son positivos y no hay ningún endpoint fallando en ninguna de las tres categorías: el diseño cumple holgadamente el período de reloj de 10 ns (100 MHz) especificado, tanto en la condición de setup como en la de hold, y también respeta los anchos mínimos de pulso de reloj/reset. Un WNS de 4.779 ns sobre un período de 10 ns indica que el camino combinacional más largo del diseño (previsiblemente, el camino a través del sumador/restador de la ALU dentro de `sum_ext/sub_ext`) consume aproximadamente 5.2 ns, muy por debajo del límite.

8. Implementación física en la Basys3

8.1. Programación vía JTAG

Tras completar síntesis e implementación, Generate Bitstream produjo el archivo `top.bit`, que contiene la configuración binaria completa de la memoria SRAM del FPGA (por ser volátil, se pierde al desenergizar la placa y debe reprogramarse en cada sesión).

En un entorno Linux, programar la placa requirió instalar previamente los drivers de cable (Digilent + Xilinx FTDI) provistos con Vivado, ejecutando como root el script `install_drivers`, que instala las reglas udev necesarias para que el sistema operativo otorgue permisos de acceso al dispositivo USB-JTAG (basado en un chip FTDI FT2232HQ) sin requerir privilegios de administrador en cada uso. Tras reconectar físicamente el cable USB, Vivado Hardware Manager detectó correctamente el dispositivo (`xc7a35t_0`) vía Auto Connect, y la programación se completó exitosamente (DONE status = 1).

8.2. Secuencia de uso funcional en la placa

- Apretar **BTNC** (reset) para partir de un estado conocido (todos los registros en 0).
- Configurar el operando A con los switches SW0 a SW7 y apretar **BTNU** para cargarlo.
- Configurar el operando B con los switches y apretar **BTNL** para cargarlo.
- Configurar el código de operación (6 bits menos significativos de los switches) y apretar **BTNR** para cargarlo.
- Apretar **BTND** para disparar el cálculo: el resultado aparece en LD0-LD7, con LD14 (carry) y LD15 (zero) reflejando las banderas correspondientes.

Esta secuencia fue verificada físicamente sobre la placa conectada, confirmando de forma directa (no solo simulada) que el mapeo de pines del XDC y la lógica de habilitación de `top.v` se comportan como lo predicho por la simulación funcional de la Sección 6.

9. Conclusiones

El desarrollo cumplió los tres requisitos centrales del enunciado: una ALU parametrizable, sintetizable e implementada exitosamente sobre la Basys3; un testbench autoverificable que combina casos dirigidos, cobertura de la rama por defecto y 500 estímulos aleatorios, con resultado 522/522 verificaciones correctas tanto en simulación comportamental como en simulación de timing post-implementación; y un análisis estático de tiempo con margen positivo en las tres métricas (WNS, WHS, WPWS), confirmando que el diseño implementado cumple sin violaciones el período de reloj de 10 ns especificado.

La verificación funcional en hardware real (secuencia de suma/resta sobre la placa física) coincidió con lo predicho por la simulación, cerrando el círculo completo del flujo de diseño digital: especificación, RTL, verificación, síntesis, implementación, análisis de tiempo, bitstream y validación física.

Mejora propuesta para trabajo futuro

Como se detalla en la Sección 6.2, una simulación de timing post-implementación completamente rigurosa requeriría que el testbench instancie `top` en lugar de `alu` directamente, aplicando estímulos a través de los puertos físicos (switches y botones) tal como lo haría un usuario real de la placa. Esto eliminaría la ambigüedad de nombre de módulo detectada, garantizando que los retardos anotados vía SDF se ejerzan efectivamente sobre el camino simulado.